
CS246: Database Management Systems Lab

Week # 09 (1 Questions, 103 Marks)

Lab session: AL1

Held on: Monday, 16-Mar-2026 (Mon)

Lab Timings: 14:00 to 17:00 Hours Pages: 5

Submission dead line: 17:00 hrs, 16-Mar-2026

Submissions accepted from: 16:45 hrs, 16-Mar-2026

Submissions accepted till: 17:30 hrs, 16-Mar-2026

Every one-minute delay in submission will result in one mark deduction

Please do not quote Network issues or technical issues after 17:30hrs

Instructors **Dr. V. Vijaya Saradhi**

Head TA **Divanshu Walia**

Department of CSE, IIT Guwahati

- This lab assignment is based on the concepts covered in chapter 3 namely nested sub-queries. Chapter 4: views, naming constraints, check constraint, type conversions from the book *Database System Concepts*, Abraham Silberschatz, Henry F. Korth, S. Sudarshan.
- **Some of the questions are so designed to yield errors.**
- Following manual files are provided along with this lab problem:
 1. Views syntax manual pages
 2. updatable and insertable views documentation
 3. views with check option.pdf

Question 1: (103 points)

Using MySQL perform the following tasks:

Task 01 (1 mark) Create a database named **week09** and create following tables in this database.

Task 02 (15 marks) Create the following tables and load data

1. (5 marks) A table **sailors** should contain 4 columns as given in the input file **sailors.csv**. Choose column names of your choice.
 - (a) (2 marks) 1st column: sailor id (primary key).
 - (b) (1 mark) 2nd column: sailor name (not null)
 - (c) (1 mark) 3rd column: rating (not null)
 - (d) (1 mark) 4th column: age (not null)
2. (4 marks) A table **boats** should contain 3 columns as given in the input file **boats.csv**. Choose column names of your choice.
 - (a) (2 marks) 1st column: boat id (primary key).

- (b) (1 mark) 2nd column: boat name (not null)
- (c) (1 mark) 3rd column: color (not null)
- 3. (6 marks) A table **reserves** should contain 3 columns as given in the input file **reserves.csv**. Choose column names of your choice.
 - (a) (1 mark) 1st column: sailor id.
 - (b) (1 mark) 2nd column: boat id
 - (c) (1 mark) 3rd column: date on which boat was reserved
 - (d) (1 mark) all the three columns form the primary key
 - (e) (2 marks) Sailor id is a foreign key pointing to sailors table; boat id is a foreign key pointing to boats table.

Task 03 (20 marks) Creating updatable views

- 1. (7 marks) Perform the following
 - (a) (3 marks) Create a view containing the columns sailor id, rating from sailors table.
 - (b) (1 mark) insert the following records into this view with the following values (91, 7), (92, 8), (93, 9), (94, 10), (22, 8)
Note: some insert operations may fail due to primary key constraints.
 - (c) (1 mark) update the rating of sid=91 to 8.
 - (d) (1 mark) delete the sailor with sid=91
 - (e) (1 mark) list (retrieve all the columns) the contents of sailors table.
- 2. (6 marks) Perform the following
 - (a) (3 marks) Create a view containing boats whose color is green. Retain all the column as part of this view.
 - (b) (1 mark) insert the following records into this view with the following values {(205, River Mania, green), (206, green-bird, green), (207, blue-warriors, blue)}
 - (c) (1 mark) Create a new view which restricts the insertion into the view with mis-matched color by adding the option **WITH CHECK OPTION**
 - (d) (1 mark) Insert the following record into this view with the following values {(207, blue-warriors, blue)}
- 3. (7 marks) Perform the following
 - (a) (3 marks) Create a view and obtain the columns **sailor id, rating, boat id, boat name**
 - (b) (1 mark) insert values (80, 8) for **sailor id, rating** into this view
 - (c) (1 mark) insert values (105, 'Lucky Lake') for **boat id, boat name** into this view
 - (d) (1 mark) update **boat name** to Jumper for **boat id = 101** in this view
 - (e) (1 mark) update **boat name** to Interlake for **boat id = 101** in this view

Task 04 (31 marks) Creating views that are not updatable

- 1. (7 marks) Perform the following

- (a) (3 marks) Create a view and obtain the columns sailor id, sailor name, rating, boat id, boat name using the query: *Find sailors whose rating equals the maximum rating among sailors who have reserved at least one boat.*
 - (b) (1 mark) Insert a record in this view with fields taking values `sailor id = 80, sailor name='best sailor', rating = 10`
 - (c) (1 mark) Update the rating of `sailor id = 74` to -9 in this view.
 - (d) (1 mark) Delete the record from this view with `sailor id` having value 74.
 - (e) (1 mark) Update the `boat name` of `boat id = 102` to Can I get updated? in this view.
2. (6 marks) Perform the following
 - (a) (3 marks) Create a view of sailors having distinct ratings and obtain the column `rating`.
 - (b) (1 mark) Insert a new rating whose value is 2 in the created view
 - (c) (1 mark) Update the rating from 7 to -7 from the created view
 - (d) (1 mark) Delete the rating 7 from the created view
3. (6 marks) Perform the following
 - (a) (3 marks) Create a view for the query: *find boats reserved by at least two sailors having the same rating.* Obtain all columns from the tables which are involved in the join operation.
 - (b) (1 mark) Insert a new record in the created view having values `sailor id = 80, sailor name='budding sailor', rating=10, age=25`
 - (c) (1 mark) Update the rating from 8 to 6 in the created view
 - (d) (1 mark) Delete the rating 7 from the created view
4. (6 marks) Perform the following
 - (a) (3 marks) Create a view for the query: Boats reserved by sailors with same rating whose age is more than 36 years. Obtain all columns from the tables which are involved in the join operation.
 - (b) (1 mark) Insert a new record in the created view having values `sailor id = 80, sailor name='budding sailor', rating=10, age=25`
Note: Observe whether the insertion succeeds and whether the row appears in the view.
 - (c) (1 mark) Update the rating from 8 to 6 in the created view
 - (d) (1 mark) Delete the rating 7 from the created view
5. (6 marks) Perform the following
 - (a) (3 marks) Create a view which contains sailors who reserved green boat or sailors who reserved blue boat. Include the following columns in the view `sailor id, sailor name, boat id, boat color` using UNION operation.
 - (b) (1 mark) Insert a new record in the created view having values `sailor id = 81, sailor name=Sanjay`
 - (c) (1 mark) Update the rating of `sailor id = 22` to 9.

(d) (1 mark) Delete the `sailor id=22` from the created view.

Task 05 (9 marks) **Creating views using views:** Perform the following

1. (3 marks) Create a view to obtain `rating` from the view in Task 03-1-(a).
2. (1 mark) Insert the following records into this view with the following values `{(7), (8), (9), (10), (8)}`
3. (1 mark) Update the rating 8 to 9.
4. (1 mark) Delete the rating 10.
5. (3 marks) Create a view to obtain `sailor id`, `boat name`, `day` using reserves and view created in Task 03-2-(a).

Task 06 (9 marks) **Effect of altering original tables on views**

1. (1 mark) Rename the column `rating` of sailors table to `rting`
2. (2 marks) List the contents all the above views that involve the column name `rating`
3. (1 mark) Rename the column `rting` of sailors to `rating`
4. (2 marks) List the contents all the above views that involve the column name `rating`
5. (1 mark) Delete the column `rating` of sailors table. Observe (that is show all records in all the created views above) how previously created views behave after modifying the schema of the base tables.
6. (2 marks) List the contents all the above views that involve the column name `rating`

Task 07 (18 = 9 + 9 marks) **Type Conversion**

1. (3 marks) Create tables `sailors_1`, `reserves_1` and `boats_1` as described in Task 02.
2. (3 marks) Populate data from `sailors.csv` into table `sailors_1`; `reserves.csv` into `reserves_1` table and `boats.csv` into `boats_1` table.
3. (3 marks) Change the following columns data type
 - (a) `sid` to `smallint`
 - (b) `bid` to `char(3)`
 - (c) `bcolor` to `char(5)`
4. (3 marks) **Inspect the foreign key constraint before schema change.** Execute the following command to inspect the definition of the `reserves` table.

```
SHOW CREATE TABLE reserves;
```

Identify the foreign key constraint referencing the `boats` table.

Now inspect the data types of the relevant columns.

```
DESCRIBE boats;
DESCRIBE reserves;
```

Observe the datatype of column `bid` in both tables.

5. (2 marks) **Modify the datatype of the primary key column in the base table.**

Change the datatype of column `bid` in the `boats` table to `CHAR(3)`.

Verify the change using:

```
DESCRIBE boats;
```

6. (1 marks) **Observe the effect on the foreign key constraint.**

Again inspect the definition of the `reserves` table.

Check whether the foreign key constraint referencing `boats(bid)` still exists and note any changes.

7. (1 mark) **Test the foreign key behaviour after the datatype change.**

Attempt to insert a new tuple into the `reserves` table.

```
INSERT INTO reserves VALUES (22, '101', '2024-01-01');
```

Observe the result.

8. (1 mark) **Make the datatype consistent in the referencing table.** Modify the datatype of column `bid` in the `reserves` table.

Verify the change using:

```
DESCRIBE reserves;
```

9. (1 mark) **Test the foreign key constraint again.**

Execute the following command once more.

```
INSERT INTO reserves VALUES (22, '101', '2024-01-01');
```

Observe whether the insertion succeeds or fails.